

16. Bölüm

Dosyalar

16.1 Dosya Nedir?

Her programın ihtiyaç duyduğu ortak görevler kullanıcıdan girdiyi (**input**) okumak ve konsolda çıktıyı (**output**) göstermektir. Bu şekilde girdi ve çıktı işlemleri yaparsak, program çalıştığı sürece veriler var olur, program sonlandırıldığında o verileri tekrar kullanamayız. Bunun için ve elektrik kesildiğinde bellekteki veriler kaybolacağından harici hafıza ortamında (**external memory**) verileri saklamak gerekir.

Dosya (file), verilerin bir araya geldiği (**collection of data**) ikincil saklama ortamıdır (**seconder storage**). Bu ikincil ortam; Bir bilgisayar hafızası (**memory**) olabileceği gibi, verilerin elektrikler kesildiğinde kaybolmayacağı sabit disk (**hard disc**), ya da bir başka ortama/bilgisayara veri gönderen (**send**) veya alan (**receive**) modem ve benzeri bir cihaz (**device**) olabilir.

16.2 Standart Dosyalar

C dili programcıya, girdi ve çıktı kısaca IO (**input-output**) için çeşitli işlevleri içeren başlık dosyaları sağlar ve tüm aygıtları (**device**) dosyalar olarak ele alınır. Bu nedenle, "ekran" gibi aygıtlar "dosyalar" ile aynı şekilde ele alınır.

Standart Dosya	Dosya Değişkeni	Aygıt
Standart Giriş	stdin	Klavye
Standart Çıktı	stdout	Ekran
Standart Hata	stderr	Kullanıcı Ekranı

Tablo 16.1: Standart Giriş-Çıkış Dosyaları

Dosyalarla işlem yapmak için `stdio.h` başlık dosyası çeşitli fonksiyonlara sahiptir ve aşağıda başlıklar halinde verilmiştir;

§16.2.1 Karakterler İçin Giriş Çıkış Fonksiyonları

```
int getchar(void);
```

Bu fonksiyon yeni satır (`'\n'`) tuşuna basmadan tek bir tuş vuruşunu yani karakteri konsoldan okur.

```
int putchar(int c);
```

Bu fonksiyon tek bir karakteri konsola yazar. Yazdırılacak karakterin ASCII kodu parametre olarak gönderilebilir.

```
#include <stdio.h>
int main() {
    char ch;
    printf("Bir karakter Giriniz: ");
    ch = getchar();
    puts("Girdiğiniz karakter: ");
    putchar(ch);
    return 0;
}
```

§16.2.2 Biçimlendirilmemiş Metin İçin Giriş Çıkış Fonksiyonları

Okuma ve yazma amacıyla dosyaya erişmek için önceden tanımlanmış bir **FILE** yapısı (**struct**) kullanır. Standart dosyalar da bu parametreye argüman olarak verilebilir.

```
char* fgets(char* str, int size, FILE* stream);
```

Bu fonksiyonda `stream` dosyasından, `str` ile kimliklendirilmiş parametre olan tampon belleğe (**buffer**), yeni satır veya dosya sonu EOF (**end-of-file**) ile karşılaşılıncaya kadar bir satırı okur. Hata yoksa `str` değişkeni, varsa EOF ya da NULL geri döndürür.

```
int fputs(const char* str, FILE* stream);
```

Bu fonksiyonda ise `str` metninin ve sonuna yeni satır karakteri koyarak `stream` dosyasına yazar. Hata yoksa yazdırılan karakter sayısını içeren pozitif bir değer döner, varsa EOF geri döndürür.

```
#include <stdio.h>
int main() {
    char adi[20];
    printf("Adınız: ");
    fgets(adi, sizeof(adi), stdin);
    fputs("Girdiğiniz Adınız", stdout);
    fputs(adi, stdout);
    return 0;
}
```

§16.2.3 Biçimlendirilmiş Metin İçin Giriş Çıkış Fonksiyonları

Şimdiye kadar çıktığı standart çıktı dosyası `stdout` standart çıkış dosyasına `printf()` fonksiyonu ile standart giriş dosyası `stdin` standart dosyasından veri girişi sağlanan `scanf()` fonksiyonlarını çokça gördük.

```
int fscanf(FILE* stream, const char* format, ...);
```

Bu fonksiyon `FILE` yapısı (**struct**) ile belirtilen dosyadan girişi sağlanan biçime göre okur. `stdin` standart giriş dosyası olarak kullanılabilir.

```
int fprintf(FILE* stream, const char* format, ...);
```

Bu fonksiyon ise `FILE` yapısı ile belirtilen dosyaya biçimlendirme metnine (**format**) göre çıktıyı üreterek yazar. `stdout` ve `stderr` standart çıkış dosyalarıdır.

Bu fonksiyonların kullanımı `scanf()` ve `printf()` fonksiyonlarına benzer. Yalnızca hangi dosyanın kullanılacağı ilk parametre olan `stream` ile belirlenir.

16.3 Standart Olmayan Dosyalar

Standart olan `stdin`, `stdout` ve `stderr` dosyaları her zaman veri alışverişine açıktır. Bu dosyalarla metinler üzerinden veri alışverişi yapılır. Standart olmayan dosyalar ise her zaman veri alışverişine açık değildir. Bu nedenle dosya ile işlem yapmadan önce dosyayı açmalı (**open**) ve işlem bittiğinde de kapatmalıyız (**close**).

§16.3.1 Dosyayı Açma ve Kapatma

Standart olmayan dosyaları açıp kapatmayı sağlayan iki fonksiyon vardır;

```
FILE *fopen(const char* file_name, const char* mode_of_operation);
```

Bu fonksiyon dosya sistemindeki adını temsil eden `file_name` ile verilen dosyayı; okumak (**read**), yazmak (**write**) ya da sonuna veri eklemek (append) için ya da bunlardan her ikisini yapmak için açar. Açılan dosyanın `FILE` yapısına (**struct**) ilişkin göstericiyi geri döndürür. Ayrıca `mode_of_operation` parametresine aşağıda verilen argümanlarla dosyayı çeşitli şekillerde işlem yapmak için açabiliriz. Bu argümanlar **dosya açma modları** (**file opening modes**) olarak adlandırılır. Bu modlar, Tablo 16.2'de verilmiştir.

Aşağıdaki fonksiyon ise açık olan dosyayı kapatır.

```
int fclose(FILE* stream);
```

Bilgi

Dosyayı sürekli açık tutmak sistem kaynaklarını meşgul etmek anlamına gelir. Dolayısıyla programcı tarafından açılan dosyayı en kısa sürede kapatmak alışkanlık haline getirilmelidir.

Aşağıda verilen örnekte görüldüğü gibi aynen konsola yazar gibi yazılması gerekenleri `fprintf()` fonksiyonu ile aynen dosyaya yazılmıştır. Kullanımı `printf()` ile aynıdır sadece ilk parametresi dosya göstericisidir. Dosya `"w+"` modunda açıldığından, program her çalıştığında dosya içeriği silerek yeniden aynı şeyleri yazacaktır.

```
#include <stdio.h>
int main()
```

Modlar	Açıklaması
"r"	Dosyayı, çalışılan klasörde/dizinde arar. Dosyayı yalnızca okumak için açar. Dosya başarıyla açılırsa fopen() onu belleğe yükler ve içindeki ilk karakteri işaret eden bir gösterici ayarlar. Dosya açılmıyorsa NULL değerini döndürür.
"w"	Dosyayı, çalışılan klasörde/dizinde arar. Dosya zaten mevcutsa içeriğinin üzerine yazılır. Dosya mevcut değilse yeni bir dosya oluşturulur. Dosya açılmıyorsa NULL değerini döndürür. Yalnızca yazmak için yeni bir dosya oluşturur.
"a"	Dosyayı çalışılan klasörde/dizinde arar. Dosya başarıyla açılırsa içeriği belleğe yükler ve içindeki son karakteri işaret eden bir gösterici ayarlar. Dosya mevcut değilse yeni bir dosya oluşturulur. Dosya açılmıyorsa NULL değerini döndürür. Dosya yalnızca dosyanın sonuna yazma yani ekleme amacıyla açılır.
"r+"	Dosyayı, çalışılan klasörde/dizinde arar. Dosyayı hem okumak hem de yazmak için açar. Başarıyla açılırsa, içeriği belleğe yükler ve içindeki ilk karakteri işaret eden bir gösterici ayarlar. Dosya açılmıyorsa NULL değerini döndürür.
"w+"	Dosyayı, çalışılan klasörde/dizinde arar. Dosya mevcutsa içeriğinin üzerine yazılır. Dosya mevcut değilse yeni bir dosya oluşturulur. Dosya açılmıyorsa NULL değerini döndürür.
"w" ve "w+" arasındaki fark, "w+" kullanılarak oluşturulan dosyayı da okuyabilmemizdir.	
"a+"	Dosyayı, çalışılan klasörde/dizinde arar. Dosya başarılı bir şekilde açılırsa içeriği belleğe yükler ve içindeki son karakteri işaret eden bir gösterici ayarlar. Dosya mevcut değilse yeni bir dosya oluşturulur. Dosya açılmıyorsa NULL değerini döndürür. Dosya okumak ve sonuna yazmak yani eklemek için açılır.

Tablo 16.2: Dosya Açma Modları

```

{
    FILE* dosyaGostericisi;
    dosyaGostericisi = fopen("metin.txt", "w+"); /* çalışılan klasörde/dizinde "metin.txt" dosyası
    ↪ yoksa oluşturulur. Varsa üzerine yazılır. Dosya, hem yazma hem okuma modunda açıldı. */
    fprintf(dosyaGostericisi, "Adi Soyadi;Yasi;Cinsiyeti\n");
    fprintf(dosyaGostericisi, "%s;%d;%c\n", "Ilhan OZKAN", 50, 'E');
    fprintf(dosyaGostericisi, "%s;%d;%c\n", "Yagmur OZKAN", 45, 'K');
    fclose(dosyaGostericisi);
    return 0;
}

```

Oluşan metin dosyası olan `metin.txt` içeriği aşağıda verilmiştir;

```

Adi Soyadi;Yasi;Cinsiyeti
Ilhan OZKAN;50;E
Yagmur OZKAN;45;K

```

Aynı dosyayı her bir satırı tam olarak okuyan `fgets()` fonksiyonu ile aşağıdaki kodla okuyabiliriz;

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define BIRSATIRDAKIENFAZLAKARAKTER 80
int main()
{
    char *dosyaAdi="metin.txt";
    char satir[BIRSATIRDAKIENFAZLAKARAKTER] = {0};
    unsigned int satirNumarasi = 0;
    FILE *dosyaGostericisi = fopen(dosyaAdi, "r");
    if (!dosyaGostericisi)
    {
        perror(dosyaAdi);
        return EXIT_FAILURE;
    }
    while (fgets(satir, BIRSATIRDAKIENFAZLAKARAKTER, dosyaGostericisi))
    {
        printf("satir[%06d]: %s", ++satirNumarasi, satir);
        if (strlen(satir) > 0 && satir[strlen(satir)-1] != '\n')
            printf("\n");
    }
}

```

```

    }
    if (fclose(dosyaGostericisi))
    {
        return EXIT_FAILURE;
        perror(dosyaAdi);
    }
}
/* Program Çıktısı:
satir[000001]: Adi Soyadi;Yasi;Cinsiyeti
satir[000002]: Ilhan OZKAN;50;E
satir[000003]: Yagmur OZKAN;45;K
*/

```

Bunların yanında `perror()` fonksiyonu, programınızda neyin yanlış gittiğini anlamana ve hata ayıklamanıza (`debug`) yardımcı olan standart hata akışına (`stderr`) açıklayıcı bir hata mesajı yazdırmak üzere tasarlanmıştır;

```

#include <stdio.h> /* perror(), fopen(), fputs() and fclose() */
#include <stdlib.h> /* EXIT_* macroları için */
int main(int argc, char **argv){
    int e = EXIT_SUCCESS;
    char *path = "output.txt";
    FILE *file = fopen(path, "w");
    if (!file) {
        perror(path);
        return EXIT_FAILURE;
    }
    if (fputs("Dosyaya Yazılacak Metin....\n", file) == EOF) {
        perror(path);
        e = EXIT_FAILURE;
    }
    if (fclose(file)) {
        perror(path);
        return EXIT_FAILURE;
    }
    return e;
}

```

Ayrıca aşağıda verilen program `popen()` aracılığıyla bir program ya da servisi çalıştırır ve işleminden gelen tüm standart çıktıyı okur ve bunu standart çıktı olan konsola yansıtır;

```

#include <stdio.h>
void print_all(FILE *stream) {
    int c;
    while ((c = getc(stream)) != EOF)
        putchar(c);
}
int main(void){
    FILE *stream;
    if ((stream = popen("dir", "r")) == NULL)
        return 1;
    print_all(stream);
    pclose(stream);
    return 0;
}

```

§16.3.2 İkili Dosyalar

Değişkenlerimizi her zaman insan gözünün okuyacağı metne çevirerek dosyalara kaydetmeyiz. Onun yerine daha az yer kaplaması açısından bellekte saklandığı şekliyle dosyaya hızlıca yazarız. Ancak bu durumda dosyayı, bellekte ikili olarak tutulan veriler gibi yazar veya okuruz. Bu dosyalar **ikili dosya** (**binary file**) olarak adlandırılır.

Bilgi

İkili dosyalar, insan gözüyle okunamazlar! Okunmaları için özel programlar gerekir.

İkili dosyalar da metin dosyaları gibi açılıp kapatılır. Ancak ikili dosyanın, dosya açma modları (**file opening modes**) farklıdır ve 16.3 tablosunda verilmiştir.

Modlar	Açıklaması
"rb"	Dosyayı çalışılan klasörde/dizinde arar. İkili dosyayı okuma modunda açar. Dosya mevcut değilse, <code>fopen()</code> fonksiyonu <code>NULL</code> değerini döndürür.
"wb"	İkili dosya yazma modunda açılır. Gösterici dosyanın başlangıcına ayarlanır ve içeriklerin üzerine ikili olarak yazılır. Dosya mevcut değilse yeni bir dosya oluşturulur.
"ab"	İkili dosya, ekleme modunda açılır. Dosya göstericisi, dosyadaki son karakterden sonra ayarlanır. Verilen dosya isminden bir dosya yoksa yeni bir dosya oluşturulur.
"rb+"	İkili dosya okuma ve yazma modunda açılır. Dosya mevcut değilse, <code>open()</code> fonksiyonu <code>NULL</code> değerini döndürür.
"wb+"	İkili dosya okuma ve yazma modunda açılır. Dosya mevcutsa içeriğin üzerine yazılır. Dosya mevcut değilse oluşturulacaktır.
"ab+"	İkili dosya okuma ve ekleme modunda açılır. Dosya mevcut değilse bir dosya oluşturulacaktır.

Tablo 16.3: İkili Dosya Açma Modları

İkili dosyalara veri okumak için kullanılan fonksiyon `fread()` fonksiyonudur;

```
size_t fread(void *ptr, size_t size, size_t nmemb, FILE *stream);
```

Bu fonksiyon;

- ♦ `stream` dosya göstericisi üzerinden veri okur ve `ptr` ile işaret edilen bellek bölgesine koyar
- ♦ `nmemb, size` ile verilen uzunluktan kaç adet okunacağını belirtir.
- ♦ Bu fonksiyon, genellikle ikili dosyaları okumak için kullanılır ancak metin dosyaları için de kullanılabilir.
- ♦ Bir okuma hatası veya dosya sonu (EOF) oluşursa `nmemb`'den az olabilen, başarıyla okunan öğelerin sayısını döndürür. `size` veya `nmemb` sıfırsa, fonksiyon sıfır döndürür ve `ptr` tarafından işaret edilen belleğin içerikleri değişmez.

İkili dosyalara veri yazmak için kullanılan fonksiyon biri `fwrite()` fonksiyonudur;

```
size_t fwrite(const void *ptr, size_t size, size_t nmemb, FILE *stream);
```

Bu fonksiyon;

- ♦ `ptr` ile işaret edilen bellekteki verileri belirtilen `stream` dosya göstericisi üzerinden dosyaya yazar.
- ♦ `nmemb, size` ile verilen uzunluktan kaç adet yazılacağını belirtir.
- ♦ Bu fonksiyon da genellikle ikili dosyalara yazmak için kullanılır ancak metin dosyaları için de kullanılabilir.
- ♦ Bu fonksiyon başarıyla yazılan toplam öğe sayısını döndürür. Bu sayı `nmemb`'den az ise bir hata oluşmuştur veya dosya sonuna ulaşılmıştır.

Aşağıda dosyaya yazılan yapı (**struct**) örneği verilmiştir. Yapılar aynen bellekte olduğu gibi dosyaya yazılır. Dosya, `"wb+"` modunda açıldığından, program her çalıştığında içeriğini silerek yeniden aynı şeyleri yazacaktır.

```
#include <stdio.h>
struct kisi {
    char adiSoyadi[16];
    int yas;
    char cinsiyet;
    float kilo;
};
typedef struct kisi Kisi;
int main() {
    FILE* dosyaGostericisi;
    Kisi kisi1={"Ilhan OZKAN",50,'E',100.0};
    Kisi kisi2={"Yagmur OZKAN",45,'K',60.0};
    int dizi[5]={1,2,3,4,5};
```



```

    for (int sayac=0; sayac<5; sayac++)
        printf("%d, ", dizi2[sayac]);
        printf("}\n");
    return 0;
}
/*Program Çıktısı:
kişi1: Adi:Ilhan OZKAN, Yas:50, Cinsiyet:E, Kilo:100.000000
kişi2: Adi:Yagmur OZKAN, Yas:45, Cinsiyet:K, Kilo:60.000000
{1,2,3,4,5,}
{1,2,3,4,5,}
*/

```

16.4 Düşün ve Çöz

- ♦ **Düşün:** Standart hata akışı (`stderr`) ile standart çıktı akışı (`stdout`) arasındaki fark nedir? Hataları neden ayrı bir akıştan göndeririz?
- ♦ **Düşün:** Metin dosyaları (`text files`) ile ikili dosyalar (`binary files`) arasındaki temel farklar nelerdir? Neden veri tabanı gibi yapılar ikili dosyaları tercih eder?
- ♦ **Düşün:** `fseek` ve `ftell` fonksiyonları, rastgele erişimli (`random access`) dosyalarda belirli bir kayda doğrudan ulaşmak için nasıl kullanılır?
- ♦ **Düşün:** Bir dosyayı "`a`" modunda açmak ile "`w`" modunda açmak arasındaki veriyi koruma farkı nedir?
- ♦ **Çöz:** Bir `struct` yapısını (örneğin bir kişi bilgisi) ikili dosya modunda dosyaya yazan ve sonra bu dosyadan geri okuyup ekrana basan bir program yazınız.
- ♦ **Çöz:** Bir metin dosyasındaki tüm karakterleri okuyup, her karakterin ASCII değerini 1 artırarak (basit bir şifreleme) başka bir dosyaya kaydeden programı yazınız.
- ♦ **Çöz:** İkili bir dosyada kayıtlı olan öğrenci yapılarını (`struct`) not ortalamasına göre filtreleyip ekrana basan kodu yazınız.
- ♦ **Çöz:** `fgetc()` ve `fputc()` fonksiyonlarını kullanarak var olan bir metin dosyasının içeriğini başka bir dosyaya kopyalayan C programını yazınız.
- ♦ **Çöz:** `fgets()` ve `fputs()` fonksiyonlarını kullanarak bir metin dosyasındaki satır sayısını sayan C programını kodlayınız.
- ♦ **Çöz:** `fprintf()` ve `fscanf()` kullanarak yapılandırılmış veri (Örn: Ad, Yaş, Not) formatında dosyaya veri yazan ve okuyan bir kod yazınız.
- ♦ **Çöz:** `fwrite()` ve `fread()` fonksiyonlarının prototiplerini açıklayınız. İkili bir dosyaya bir `struct Öğrenci` yapısını doğrudan bayt blokları halinde yazıp okuyan programı geliştiriniz.
- ♦ **Çöz:** `fseek()`, `ftell()` ve `rewind()` fonksiyonlarının görevlerini açıklayınız. Bir dosyanın boyutunu (bayt cinsinden) `fseek()` ve `ftell()` kullanarak bulan C fonksiyonunu yazınız.
- ♦ **Çöz:** Döngü şartı olarak `while (!feof(file))` yazmanın neden hatalı okumalara (son veriyi iki kez okuma) yol açabileceğini, doğru okuma döngüsü kalıpları üzerinden açıklayınız.
- ♦ **Çöz:** İkili olarak kaydedilmiş öğrenci kayıtları dosyasında, id'si verilen öğrencinin notunu dosyanın tamamını belleğe almadan doğrudan ilgili kayda konumlanarak (`fseek()`) güncelleyen fonksiyonu yazınız.
- ♦ **Çöz:** Bir metin dosyasında geçen belirli bir kelimeyi (Örn: "C") arayıp yerine başka bir kelime (Örn: "C++") yazarak dosyayı güncelleyen C programını geliştiriniz.